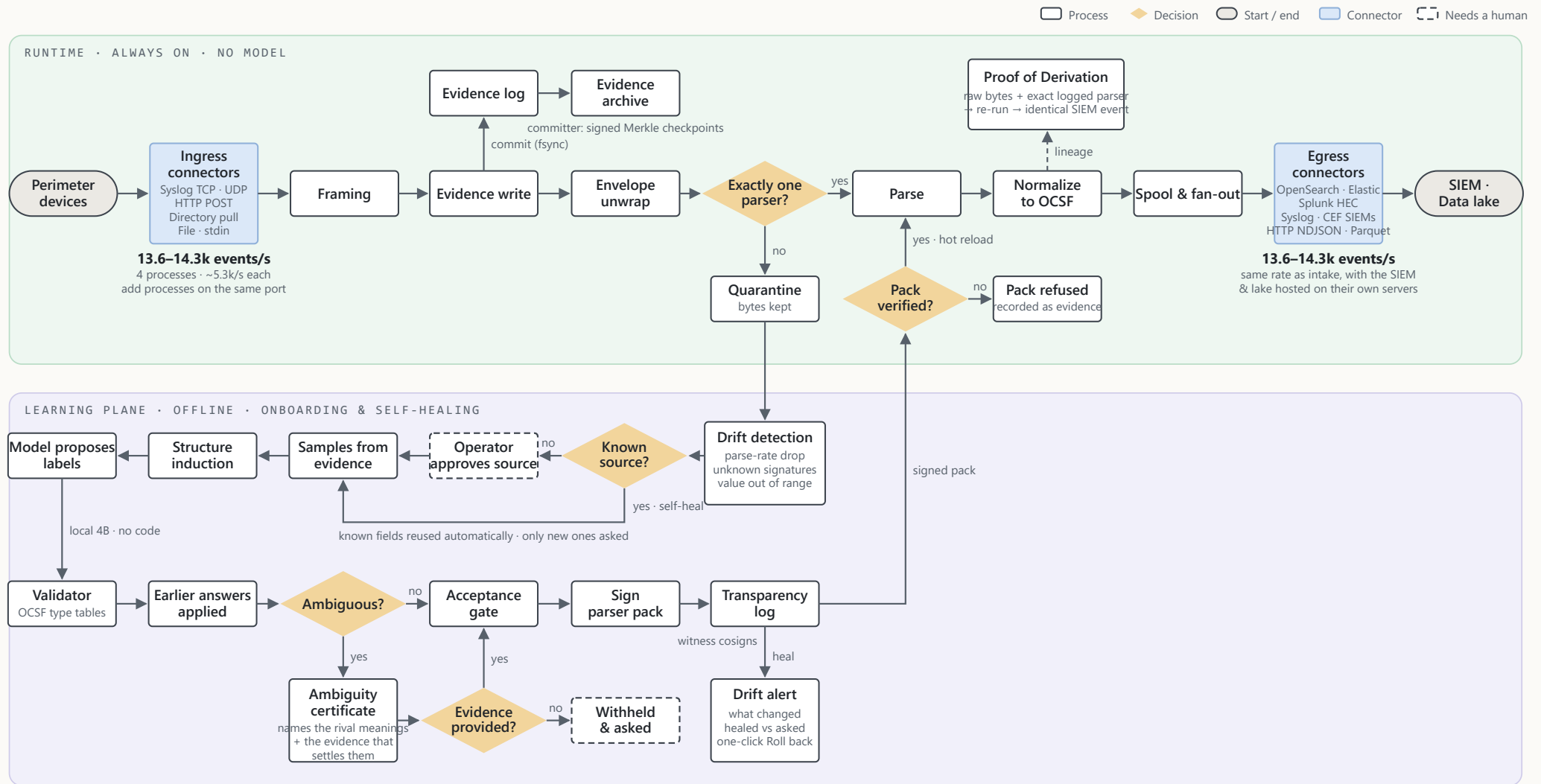


ULPF · Architecture flow



The top band runs all the time and holds no model. The bottom band runs only when a new or changed format appears. Figures measured on one desktop (Ryzen 7 2700X, 8 cores), evidence log on, exactly-once checked.

How ULPF works, and why it is built this way

Two halves: a fast, always-on runtime that never guesses, and an offline learning plane that teaches it new formats.

The runtime: every log, every time

A firewall or IDS sends its logs in over syslog (TCP or UDP), HTTP, or files. ULPF cuts the stream into individual events, and **before it reads anything, it writes the original bytes to the evidence log** and commits them to disk. A separate committer seals that log with signed Merkle checkpoints and ships it to the evidence archive.

Only then does ULPF look at the event. It strips the syslog, CEF or LEEF wrapper and recognises the format by its shape. If exactly one parser fits, the event is parsed and turned into **OCSF**, one common schema for every vendor. If none fits, or more than one could, the event is **quarantined** with its bytes kept. ULPF never guesses.

Every normalized event carries its lineage: the hash of its raw bytes, where they are stored, and which parser version made it. That is what **Proof of Derivation** uses: it re-runs that exact parser on the original bytes and shows the result equals what the SIEM holds. Finally, events go into a spool with one cursor per destination, so the SIEM and the data lake each receive everything, at their own pace, even after an outage.

The learning plane: new and changed formats

Quarantined lines are watched. A new source waits for an operator's approval; a known device that changed its format starts healing on its own. ULPF takes samples from the evidence log, works out the structure, and a small local model (no internet) suggests what each field means. It suggests only; a validator checks every suggestion against the OCSF type tables, and answers already given for that device are reused.

When the samples can't decide something, such as which address is the source, ULPF issues an **ambiguity certificate** and asks for the one piece of evidence that settles it. Anything nobody can confirm is held back and asked, never guessed. The approved parser is signed and added to the **transparency log**; the runtime checks it and loads it without a restart. A changed format comes with a drift alert and a one-click roll back.

What this design gives you

- **Nothing is lost.** Every original byte is stored before parsing, hashed, sealed and archived. Vendor fields with no OCSF home are kept by name.
- **Every result can be proven.** Any SIEM event traces back to its exact original bytes and the exact parser that produced it, and the check runs offline. ULPF can draft the certificate Indian courts ask for (BSA 2023 §63(4)).
- **No hand-written parsers.** The model drafts, a person answers only what the data can't tell. A FortiGate source took 12 decisions instead of 73 by hand.
- **It heals itself.** When a real FortiGate switched to JSON, 45–47 of about 52 fields carried over automatically and only 6–7 new ones were asked.
- **No silent mistakes.** Unclear events are quarantined, unclear fields are asked, and the runtime refuses any parser that isn't signed and logged.
- **Fast and predictable.** No model at run time and only linear-time regex, so one bad line can't stall it. One process handles ~5,300 events/s with the evidence log on; four held 13.6–14.3k/s, above the 11,574/s that one billion events a day needs.
- **One view of every vendor.** One query for an attacker's address finds both the firewall's denials and the IDS's alerts, whatever each vendor calls the field.
- **Fits a secure network.** Everything runs offline and in containers, and processes can be added or removed while events flow.

Tested with a licensed FortiGate VM (FortiOS 7.4.12) and Suricata 7.0.7. OpenSearch and the Parquet lake were tested end to end; the Splunk HEC and CEF outputs were tested against stand-ins only.